这次我们逐步的分析ELF文件每一个结构,结合一个Demo。分别编译成可执行文件和可重定位文件。

```c
#include <stdio.h>

int main()
{
    int i=1;
    printf("%d\n",i);
    return i;
}
```

ELF Header

```
reticence@reticence-PC:~/code/c$ readelf -SlhW hello.out
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x401040
  Start of program headers:          64 (bytes into file)
  Start of section headers:          14560 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         11
  Size of section headers:           64 (bytes)
  Number of section headers:         29
  Section header string table index: 28
```

- Magic Number：魔数，表示该文件是ELF文件。
- Entry point address：ELF程序的入口虚拟地址。操作系统在加载完该程序之后，从这个地址开始执行进程的指令。该文件是REL可重定位文件，所以没有所谓的入口地址。
- Start of section headers：Section Header的文件偏移位置。在上面的例子也就是说ELF文件的第1104偏移开始是Section Header。
- Start of program headers：Program Header的文件偏移位置。

Section Header

链接之前的:

```
Section Headers:
  [Nr] Name              Type             Address           Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL             0000000000000000 000000 000000 00      0   0  0
  [ 1] .text             PROGBITS         0000000000000000 000040 000028 00  AX  0   0  1
  [ 2] .rela.text        RELA             0000000000000000 0001e8 000030 18   I 10   1  8
  [ 3] .data             PROGBITS         0000000000000000 000068 000000 00  WA  0   0  1
  [ 4] .bss              NOBITS           0000000000000000 000068 000000 00  WA  0   0  1
  [ 5] .rodata           PROGBITS         0000000000000000 000068 000004 00   A  0   0  1
  [ 6] .comment          PROGBITS         0000000000000000 00006c 000024 01  MS  0   0  1
  [ 7] .note.GNU-stack   PROGBITS         0000000000000000 000090 000000 00      0   0  1
  [ 8] .eh_frame         PROGBITS         0000000000000000 000090 000038 00   A  0   0  8
  [ 9] .rela.eh_frame    RELA             0000000000000000 000218 000018 18   I 10   8  8
  [10] .symtab           SYMTAB           0000000000000000 0000c8 000108 18     11   9  8
  [11] .strtab           STRTAB           0000000000000000 0001d0 000015 00      0   0  1
  [12] .shstrtab         STRTAB           0000000000000000 000230 000061 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  l (large), p (processor specific)
```

链接之后的:

```
Section Headers:
  [Nr] Name              Type             Address           Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL             0000000000000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS         00000000004002a8 0002a8 00001c 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE             00000000004002c4 0002c4 000020 00   A  0   0  4
  [ 3] .note.gnu.build-id NOTE             00000000004002e4 0002e4 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH         0000000000400308 000308 00001c 00   A  5   0  8
  [ 5] .dynsym           DYNSYM           0000000000400328 000328 000060 18   A  6   1  8
  [ 6] .dynstr           STRTAB           0000000000400388 000388 00003f 00   A  0   0  1
  [ 7] .gnu.version      VERSYM           00000000004003c8 0003c8 000008 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED          00000000004003d0 0003d0 000020 00   A  6   1  8
  [ 9] .rela.dyn         RELA             00000000004003f0 0003f0 000030 18   A  5   0  8
  [10] .rela.plt         RELA             0000000000400420 000420 000018 18  AI  5  22  8
  [11] .init             PROGBITS         0000000000401000 001000 000017 00  AX  0   0  4
  [12] .plt              PROGBITS         0000000000401020 001020 000020 10  AX  0   0 16
  [13] .text             PROGBITS         0000000000401040 001040 000171 00  AX  0   0 16
  [14] .fini             PROGBITS         00000000004011b4 0011b4 000009 00  AX  0   0  4
  [15] .rodata           PROGBITS         0000000000402000 002000 000008 00   A  0   0  4
  [16] .eh_frame_hdr     PROGBITS         0000000000402008 002008 00003c 00   A  0   0  4
  [17] .eh_frame         PROGBITS         0000000000402048 002048 000100 00   A  0   0  8
  [18] .init_array       INIT_ARRAY       0000000000403e10 002e10 000008 08  WA  0   0  8
  [19] .fini_array       FINI_ARRAY       0000000000403e18 002e18 000008 08  WA  0   0  8
  [20] .dynamic          DYNAMIC          0000000000403e20 002e20 0001d0 10  WA  6   0  8
  [21] .got              PROGBITS         0000000000403ff0 002ff0 000010 08  WA  0   0  8
  [22] .got.plt          PROGBITS         0000000000404000 003000 000020 08  WA  0   0  8
  [23] .data             PROGBITS         0000000000404020 003020 000010 00  WA  0   0  8
  [24] .bss              NOBITS           0000000000404030 003030 000008 00  WA  0   0  1
  [25] .comment          PROGBITS         0000000000000000 003030 000023 01  MS  0   0  1
  [26] .symtab           SYMTAB           0000000000000000 003058 0005b8 18     27  43  8
  [27] .strtab           STRTAB           0000000000000000 003610 0001cb 00      0   0  1
  [28] .shstrtab         STRTAB           0000000000000000 0037db 000103 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  l (large), p (processor specific)
```

- Name：段的名字。

- TYPE：段的类型，STRTAB字符串表，SYMTAB符号表。
- Address：载入内存的虚拟地址。
- Offset：在二进制文件内的偏移。
- Size：段的大小
- Align：对齐。总是2的幂次。

Program Header

Section和Segment有什么区别呢？在中文资料里经常将他们混淆（段和节）。但实际上，
我们在Section Header列出的项目是Section，Program Header列出的项目是Segment。
从**链接**的角度看，ELF文件是按照Section来组织的。
从**装载**的角度看，ELF文件是按照Segment来组织的。也就是说，同一个Segment会被映射到同一块内存区域。一般来说有**相同权限**的Section（我发现有的不同权限的也会被载入一个段里）会被映射到同一个Segment。
当然有些Section是不会被映射到Segment的，也就是不载入内存的。

符号表SYMTAB

参考《程序员的自我修养》P83。

```
typedef struct {
    uint32_t      st_name;
    unsigned char st_info;
    unsigned char st_other;
    uint16_t      st_shndx;
    Elf64_Addr    st_value;
    uint64_t      st_size;
} Elf64_Sym;
```

```
Symbol table '.symtab' contains 11 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS hello.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1
     3: 0000000000000000     0 SECTION LOCAL  DEFAULT    3
     4: 0000000000000000     0 SECTION LOCAL  DEFAULT    4
     5: 0000000000000000     0 SECTION LOCAL  DEFAULT    5
     6: 0000000000000000     0 SECTION LOCAL  DEFAULT    7
     7: 0000000000000000     0 SECTION LOCAL  DEFAULT    8
     8: 0000000000000000     0 SECTION LOCAL  DEFAULT    6
     9: 0000000000000000    40 FUNC    GLOBAL DEFAULT    1 main
    10: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND printf
```

- name：其实存储的是符号名在字符串表的下标。
- size：是符号的大小，比如double是8字节。如果是0的话，可能是0或未知。
- info：符号类型和绑定信息Type和Bind。
- shndx：符号所在的段。如果符号在本目标文件，就是section table的下标。但是如果不在本文件的段中，就是别的特殊常量。比如文件名就是ABS一个绝对的值。再比如main，在代码段.text，所以是1。
- value：符号对应的值。

强符号和弱符号

对于C语言来说。编译器默认函数和初始化了的全局变量为强符号，未初始化的全局变量为弱符号。

- 强符号不允许被多次定义。
- 如果一个符号在一个目标文件是强符号，在其他文件是弱符号，选择强符号。
- 如果一个符号在所有的目标文件都是弱符号，那么选择其中占用空间最大的。

重定位表

```
hello.o:     file format elf64-x86-64

RELOCATION RECORDS FOR [.text]:
OFFSET           TYPE              VALUE
0000000000000015 R_X86_64_32       .rodata
000000000000001f R_X86_64_PLT32    printf-0x0000000000000004


RELOCATION RECORDS FOR [.eh_frame]:
OFFSET           TYPE              VALUE
0000000000000020 R_X86_64_PC32     .text
```

TYPE表示不同的重定位方式。

OFFSET表示需要重定位的符号的**第一个字节**相对于**段起始**的偏移。

重定位发生在链接的时候，我们需要把之前没有重定位的“虚假地址”换成真正的地址。

链接前后：

```
0000000000000000 <main>:
   0:   55                      push   %rbp
   1:   48 89 e5                mov    %rsp,%rbp
   4:   48 83 ec 10             sub    $0x10,%rsp
   8:   c7 45 fc 01 00 00 00    movl   $0x1,-0x4(%rbp)
   f:   8b 45 fc                mov    -0x4(%rbp),%eax
  12:   89 c6                   mov    %eax,%esi
  14:   bf 00 00 00 00          mov    $0x0,%edi
  19:   b8 00 00 00 00          mov    $0x0,%eax
  1e:   e8 00 00 00 00          callq  23 <main+0x23>
  23:   8b 45 fc                mov    -0x4(%rbp),%eax
  26:   c9                      leaveq
  27:   c3                      retq
```

```
0000000000401122 <main>:
  401122:       55                      push   %rbp
  401123:       48 89 e5                mov    %rsp,%rbp
  401126:       48 83 ec 10             sub    $0x10,%rsp
  40112a:       c7 45 fc 01 00 00 00    movl   $0x1,-0x4(%rbp)
  401131:       8b 45 fc                mov    -0x4(%rbp),%eax
  401134:       89 c6                   mov    %eax,%esi
  401136:       bf 04 20 40 00          mov    $0x402004,%edi
  40113b:       b8 00 00 00 00          mov    $0x0,%eax
  401140:       e8 eb fe ff ff          callq  401030 <printf@plt>
  401145:       8b 45 fc                mov    -0x4(%rbp),%eax
  401148:       c9                      leaveq
  401149:       c3                      retq
  40114a:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
```

字符串表STRTAB

```
000001d0: 00 68 65 6C 6C 6F 2E 63 00 6D 61 69 6E 00 70 72    .hello.c.main.pr
000001e0: 69 6E 74 66 00 00 00 00 15 00 00 00 00 00 00 00    intf............
```

```
00000230: 00 2E 73 79 6D 74 61 62 00 2E 73 74 72 74 61 62    ..symtab..strtab
00000240: 00 2E 73 68 73 74 72 74 61 62 00 2E 72 65 6C 61    ..shstrtab..rela
00000250: 2E 74 65 78 74 00 2E 64 61 74 61 00 2E 62 73 73    .text..data..bss
00000260: 00 2E 72 6F 64 61 74 61 00 2E 63 6F 6D 6D 65 6E    ..rodata..commen
00000270: 74 00 2E 6E 6F 74 65 2E 47 4E 55 2D 73 74 61 63    t..note.GNU-stac
00000280: 6B 00 2E 72 65 6C 61 2E 65 68 5F 66 72 61 6D 65    k..rela.eh_frame
00000290: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
```

我们在上面可以看到.strtab段和.shstrtab段的类型都是字符串表。我们来看他的构造是怎么样的。以链接之前的为例：

.strtab在文件偏移的0x1d0处，长度为0x15。**每一个字符串都是C字符串，以`'\0'`结束**。虽然长度是0x15，但是由于对齐所以下一个段在0x1e8才开始。

.shstrtab在文件偏移的0x230处，长度为0x61。可以看到段的名字存储在这里。